SOUTHEAST UNIVERSITY
CSE384 – Database Design Lab
PROJECT SQL QUERIES SUBMISSION

Do not rename, remove or reorder the section headings below. Replace the bracketed placeholders only. Submission is evaluated automatically and then reviewed.

Section 1: Student Information

Field	Details
Student Name	[your full name]
Student ID	[your ID]
Section	[section]
Group ID	[group number, or 'Individual']
Project Title	DropX – Parcel Delivery & Logistics Management System
DBMS Used	MySQL 8.0+

Section 2: Peer Review and Contribution

List every member of your group (including yourself) and the percentage of total project work each member contributed. The total must equal 100%. Every member submits this table independently, so answers are cross-checked.

Member Name	Student ID	Work Done (1 line)	Contribution %
[name]	[ID]	[e.g. designed schema, wrote report queries]	[%]
[name]	[ID]	[...]	[%]
[name]	[ID]	[...]	[%]
[name]	[ID]	[...]	[%]
TOTAL			**100%**

Peer comment (2–3 lines, honest feedback on the teamwork, including any issue you faced):

[write here]

Self-declaration: I confirm the information above is accurate and the queries below are from our own project.

Section 3: Database Schema

Paste the complete CREATE TABLE statements (with all constraints) and sample INSERT statements, so the queries below can be re-run exactly as written.

Complete CREATE TABLE statements

```
-- Complete DropX MySQL 8.0+ table schema.
SET NAMES utf8mb4;
SET FOREIGN_KEY_CHECKS = 0;

CREATE TABLE IF NOT EXISTS roles (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    description VARCHAR(255) NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
```

```
    UNIQUE KEY uq_roles_name (name)
);

CREATE TABLE IF NOT EXISTS branches (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    name VARCHAR(150) NOT NULL,
    code VARCHAR(50) NOT NULL,
    phone VARCHAR(30) NULL,
    address VARCHAR(500) NULL,
    city VARCHAR(100) NULL,
    district VARCHAR(100) NULL,
    latitude DECIMAL(10,7) NULL,
    longitude DECIMAL(10,7) NULL,
    status ENUM('ACTIVE','INACTIVE') NOT NULL DEFAULT 'ACTIVE',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uq_branches_code (code),
    KEY idx_branches_status (status),
    KEY idx_branches_district (district)
);

CREATE TABLE IF NOT EXISTS hubs (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    branch_id BIGINT UNSIGNED NOT NULL,
    name VARCHAR(150) NOT NULL,
    code VARCHAR(50) NOT NULL,
    address VARCHAR(500) NULL,
    district VARCHAR(100) NULL,
    latitude DECIMAL(10,7) NULL,
    longitude DECIMAL(10,7) NULL,
    capacity INT UNSIGNED NULL,
    status ENUM('ACTIVE','INACTIVE','MAINTENANCE') NOT NULL DEFAULT 'ACTIVE',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uq_hubs_code (code),
    KEY idx_hubs_branch_id (branch_id),
    KEY idx_hubs_status (status),
    KEY idx_hubs_district (district),
    CONSTRAINT fk_hubs_branch
        FOREIGN KEY (branch_id) REFERENCES branches(id)
        ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE IF NOT EXISTS hub_services (
    hub_id BIGINT UNSIGNED NOT NULL,
    service ENUM('PICKUP','SORTING','DELIVERY') NOT NULL,
    PRIMARY KEY (hub_id, service),
    KEY idx_hub_services_service_hub (service, hub_id),
    CONSTRAINT fk_hub_services_hub
        FOREIGN KEY (hub_id) REFERENCES hubs(id)
        ON UPDATE CASCADE ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS users (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    branch_id BIGINT UNSIGNED NULL,
    name VARCHAR(150) NOT NULL,
    email VARCHAR(255) NOT NULL,
    phone VARCHAR(30) NULL,
    password_hash VARCHAR(255) NOT NULL,
    must_change_password BOOLEAN NOT NULL DEFAULT FALSE,
    status ENUM('ACTIVE','INACTIVE','SUSPENDED') NOT NULL DEFAULT 'ACTIVE',
    avatar_url VARCHAR(500) NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uq_users_email (email),
    KEY idx_users_branch_id (branch_id),
    KEY idx_users_phone (phone),
    KEY idx_users_status (status),
    CONSTRAINT fk_users_branch
        FOREIGN KEY (branch_id) REFERENCES branches(id)
        ON UPDATE CASCADE ON DELETE SET NULL
);

CREATE TABLE IF NOT EXISTS user_roles (
    user_id BIGINT UNSIGNED NOT NULL,
    role_id BIGINT UNSIGNED NOT NULL,
    PRIMARY KEY (user_id, role_id),
    CONSTRAINT fk_user_roles_user
        FOREIGN KEY (user_id) REFERENCES users(id)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_user_roles_role
        FOREIGN KEY (role_id) REFERENCES roles(id)
        ON UPDATE CASCADE ON DELETE CASCADE
```

```
);

CREATE TABLE IF NOT EXISTS role_permissions (
    role_id BIGINT UNSIGNED NOT NULL,
    permission_key VARCHAR(150) NOT NULL,
    PRIMARY KEY (role_id, permission_key),
    CONSTRAINT fk_role_permissions_role
        FOREIGN KEY (role_id) REFERENCES roles(id)
        ON UPDATE CASCADE ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS user_hubs (
    user_id BIGINT UNSIGNED NOT NULL,
    hub_id BIGINT UNSIGNED NOT NULL,
    PRIMARY KEY (user_id, hub_id),
    KEY idx_user_hubs_hub_id (hub_id),
    CONSTRAINT fk_user_hubs_user
        FOREIGN KEY (user_id) REFERENCES users(id)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_user_hubs_hub
        FOREIGN KEY (hub_id) REFERENCES hubs(id)
        ON UPDATE CASCADE ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS customers (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    -- Public reference (`CUS-0001`), assigned by the server from `sequences`.
    -- Separate from `id` so the number shown to staff stays stable and dense
    -- even if rows are ever re-keyed. Default '' exists only so the guarded
    -- upgrade statement below can add this column to a populated table; every
    -- insert supplies a real value.
    code VARCHAR(20) NOT NULL DEFAULT '',
    name VARCHAR(150) NOT NULL,
    phone VARCHAR(30) NOT NULL,
    email VARCHAR(255) NULL,
    type ENUM('INDIVIDUAL','BUSINESS') NOT NULL DEFAULT 'INDIVIDUAL',
    -- TEMP: created on OTP request after consent; ACTIVE: after OTP verified
    status ENUM('TEMP','ACTIVE') NOT NULL DEFAULT 'TEMP',
    -- Public URL of the customer's uploaded avatar; NULL means the initials
    -- fallback in the UI. The bytes live in object storage, never MySQL.
    avatar_url VARCHAR(500) NULL,
    consent_accepted_at DATETIME NULL,
    activated_at DATETIME NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uq_customers_code (code),
    UNIQUE KEY uq_customers_phone (phone),
    UNIQUE KEY uq_customers_email (email),
    KEY idx_customers_type (type),
    KEY idx_customers_status (status)
);

CREATE TABLE IF NOT EXISTS customer_addresses (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    customer_id BIGINT UNSIGNED NOT NULL,
    label VARCHAR(50) NULL,
    -- Structured to match the booking cascade: a saved address carries the same
    -- city/zone/area ids a booking end does, so it can prefill the picker. The
    -- old free-text `city`/`district`/`postal_code` columns are gone.
    city_id BIGINT UNSIGNED NOT NULL,
    zone_id BIGINT UNSIGNED NOT NULL,
    area_id BIGINT UNSIGNED NULL,
    address_line VARCHAR(300) NOT NULL,
    landmark VARCHAR(255) NULL,
    latitude DECIMAL(10,7) NULL,
    longitude DECIMAL(10,7) NULL,
    is_default BOOLEAN NOT NULL DEFAULT FALSE,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    KEY idx_customer_addresses_customer_id (customer_id),
    KEY idx_customer_addresses_city (city_id),
    KEY idx_customer_addresses_zone (zone_id),
    KEY idx_customer_addresses_area (area_id),
    CONSTRAINT fk_customer_addresses_customer
        FOREIGN KEY (customer_id) REFERENCES customers(id)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_customer_addresses_city
        FOREIGN KEY (city_id) REFERENCES service_cities(id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_customer_addresses_zone
        FOREIGN KEY (zone_id) REFERENCES service_zones(id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_customer_addresses_area
        FOREIGN KEY (area_id) REFERENCES service_areas(id)
        ON UPDATE CASCADE ON DELETE SET NULL
```

```
);

CREATE TABLE IF NOT EXISTS service_cities (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    code VARCHAR(50) NOT NULL,
    -- ISD / SUBURB / OSD is what selects a pricing lane, so it is a column of
    -- the city rather than a property of a quote.
    service_type ENUM('ISD','SUBURB','OSD') NOT NULL DEFAULT 'ISD',
    status ENUM('ACTIVE','INACTIVE') NOT NULL DEFAULT 'ACTIVE',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uq_service_cities_code (code),
    KEY idx_service_cities_status (status),
    KEY idx_service_cities_service_type (service_type)
);

CREATE TABLE IF NOT EXISTS service_zones (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    city_id BIGINT UNSIGNED NOT NULL,
    name VARCHAR(100) NOT NULL,
    code VARCHAR(50) NOT NULL,
    status ENUM('ACTIVE','INACTIVE') NOT NULL DEFAULT 'ACTIVE',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    -- Scoped to the city, not global: two cities may each have a "Sadar".
    UNIQUE KEY uq_service_zones_city_code (city_id, code),
    KEY idx_service_zones_city (city_id),
    KEY idx_service_zones_status (status),
    CONSTRAINT fk_service_zones_city
        FOREIGN KEY (city_id) REFERENCES service_cities(id)
        ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE IF NOT EXISTS service_areas (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    zone_id BIGINT UNSIGNED NOT NULL,
    name VARCHAR(100) NOT NULL,
    code VARCHAR(50) NOT NULL,
    status ENUM('ACTIVE','INACTIVE') NOT NULL DEFAULT 'ACTIVE',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uq_service_areas_zone_code (zone_id, code),
    KEY idx_service_areas_zone (zone_id),
    KEY idx_service_areas_status (status),
    CONSTRAINT fk_service_areas_zone
        FOREIGN KEY (zone_id) REFERENCES service_zones(id)
        ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE IF NOT EXISTS pricing_lanes (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    pickup_type ENUM('ISD','SUBURB','OSD','ISD_ON_DEMAND') NOT NULL,
    delivery_type ENUM('ISD','SUBURB','OSD','SAME_CITY','DIFFERENT_CITY','SAME_CITY_ON_DEMAND') NOT NULL,
    -- Redundant with `delivery_type = 'SAME_CITY'` by construction, kept as a
    -- column because it is half of the lane's identity and the quote reads it.
    same_city TINYINT(1) NOT NULL DEFAULT 0,
    status ENUM('ACTIVE','INACTIVE') NOT NULL DEFAULT 'ACTIVE',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uq_pricing_lanes_route (pickup_type, delivery_type, same_city),
    KEY idx_pricing_lanes_status (status)
);

CREATE TABLE IF NOT EXISTS pricing_slabs (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    pricing_lane_id BIGINT UNSIGNED NOT NULL,
    -- Grams, not kilograms: the matrix bands are 0-200g, 201-500g, 501g-1kg,
    -- 1kg-2kg, and a kilogram column could not express the first two.
    min_weight_grams INT UNSIGNED NOT NULL,
    max_weight_grams INT UNSIGNED NOT NULL,
    base_fee DECIMAL(12,2) NOT NULL DEFAULT 0,
    -- Charged per whole kg *above* `max_weight_grams`, which is how a parcel
    -- over 2kg is priced: the top slab's base plus its extra-weight rule. It is
    -- an explicit per-lane number, not the old `price_per_kg` carry-over.
    extra_kg_fee DECIMAL(12,2) NOT NULL DEFAULT 0,
    cod_percentage DECIMAL(5,2) NOT NULL DEFAULT 0,
    cod_fixed_fee DECIMAL(12,2) NOT NULL DEFAULT 0,
    status ENUM('ACTIVE','INACTIVE') NOT NULL DEFAULT 'ACTIVE',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    -- Two slabs of one lane cannot share a boundary, which is the structural
```

```
    -- half of the no-overlap rule. Full overlap detection needs a row comparison
    -- and lives in `pricing-lanes.service`, which rejects an insert or an edit
    -- whose range intersects an existing slab of the same lane.
    UNIQUE KEY uq_pricing_slabs_lane_min (pricing_lane_id, min_weight_grams),
    UNIQUE KEY uq_pricing_slabs_lane_max (pricing_lane_id, max_weight_grams),
    KEY idx_pricing_slabs_lane_status (pricing_lane_id, status),
    CONSTRAINT fk_pricing_slabs_lane
        FOREIGN KEY (pricing_lane_id) REFERENCES pricing_lanes(id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT chk_pricing_slabs_range
        CHECK (max_weight_grams > min_weight_grams)
);

CREATE TABLE IF NOT EXISTS vehicles (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    registration_number VARCHAR(50) NOT NULL,
    type ENUM('BIKE','VAN','TRUCK','COVERED_VAN') NOT NULL,
    capacity_kg DECIMAL(10,2) NOT NULL DEFAULT 0,
    status ENUM('AVAILABLE','IN_USE','MAINTENANCE','INACTIVE') NOT NULL DEFAULT 'AVAILABLE',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uq_vehicles_registration_number (registration_number),
    KEY idx_vehicles_status (status)
);

CREATE TABLE IF NOT EXISTS routes (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    name VARCHAR(150) NOT NULL,
    code VARCHAR(50) NOT NULL,
    origin_hub_id BIGINT UNSIGNED NOT NULL,
    destination_hub_id BIGINT UNSIGNED NOT NULL,
    distance_km DECIMAL(10,2) NULL,
    estimated_minutes INT UNSIGNED NULL,
    status ENUM('ACTIVE','INACTIVE') NOT NULL DEFAULT 'ACTIVE',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uq_routes_code (code),
    KEY idx_routes_origin_hub (origin_hub_id),
    KEY idx_routes_destination_hub (destination_hub_id),
    KEY idx_routes_status (status),
    CONSTRAINT fk_routes_origin_hub
        FOREIGN KEY (origin_hub_id) REFERENCES hubs(id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_routes_destination_hub
        FOREIGN KEY (destination_hub_id) REFERENCES hubs(id)
        ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE IF NOT EXISTS route_stops (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    route_id BIGINT UNSIGNED NOT NULL,
    hub_id BIGINT UNSIGNED NOT NULL,
    sequence_no INT UNSIGNED NOT NULL,
    estimated_arrival_minutes INT UNSIGNED NULL,
    PRIMARY KEY (id),
    UNIQUE KEY uq_route_stops_sequence (route_id, sequence_no),
    UNIQUE KEY uq_route_stops_hub (route_id, hub_id),
    KEY idx_route_stops_hub_id (hub_id),
    CONSTRAINT fk_route_stops_route
        FOREIGN KEY (route_id) REFERENCES routes(id)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_route_stops_hub
        FOREIGN KEY (hub_id) REFERENCES hubs(id)
        ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE IF NOT EXISTS riders (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    user_id BIGINT UNSIGNED NOT NULL,
    hub_id BIGINT UNSIGNED NOT NULL,
    employee_code VARCHAR(50) NOT NULL,
    license_number VARCHAR(100) NULL,
    -- How the company pays the rider (ops/payroll; COD cash still goes to company)
    compensation_type ENUM('SALARIED','CONTRACTUAL','COMMISSION','MIXED')
        NOT NULL DEFAULT 'SALARIED',
    status ENUM('AVAILABLE','BUSY','OFFLINE','SUSPENDED') NOT NULL DEFAULT 'OFFLINE',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uq_riders_user_id (user_id),
    UNIQUE KEY uq_riders_employee_code (employee_code),
    KEY idx_riders_hub_id (hub_id),
    KEY idx_riders_status (status),
    CONSTRAINT fk_riders_user
        FOREIGN KEY (user_id) REFERENCES users(id)
```

```
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_riders_hub
        FOREIGN KEY (hub_id) REFERENCES hubs(id)
        ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE IF NOT EXISTS rider_applications (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    name VARCHAR(150) NOT NULL,
    phone VARCHAR(30) NOT NULL,
    email VARCHAR(255) NULL,
    district VARCHAR(100) NOT NULL,
    vehicle_type ENUM('BICYCLE','MOTORCYCLE','CAR','VAN','OTHER') NOT NULL,
    license_number VARCHAR(100) NULL,
    experience_years DECIMAL(4,1) NULL,
    availability VARCHAR(100) NOT NULL,
    notes VARCHAR(1000) NULL,
    status ENUM('PENDING','REVIEWING','APPROVED','REJECTED') NOT NULL DEFAULT 'PENDING',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    KEY idx_rider_applications_status_created (status, created_at),
    KEY idx_rider_applications_phone (phone)
);

CREATE TABLE IF NOT EXISTS rider_locations (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    rider_id BIGINT UNSIGNED NOT NULL,
    latitude DECIMAL(10,7) NOT NULL,
    longitude DECIMAL(10,7) NOT NULL,
    recorded_at DATETIME NOT NULL,
    PRIMARY KEY (id),
    KEY idx_rider_locations_rider_time (rider_id, recorded_at),
    CONSTRAINT fk_rider_locations_rider
        FOREIGN KEY (rider_id) REFERENCES riders(id)
        ON UPDATE CASCADE ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS parcels (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    tracking_number VARCHAR(50) NOT NULL,
    sender_customer_id BIGINT UNSIGNED NOT NULL,
    -- The receiver may or may not be a DropX account holder.
    receiver_customer_id BIGINT UNSIGNED NULL,
    receiver_name VARCHAR(150) NOT NULL,
    receiver_phone VARCHAR(30) NOT NULL,
    receiver_secondary_phone VARCHAR(30) NULL,
    origin_hub_id BIGINT UNSIGNED NOT NULL,
    destination_hub_id BIGINT UNSIGNED NOT NULL,
    current_hub_id BIGINT UNSIGNED NULL,
    weight DECIMAL(10,2) NOT NULL,
    length DECIMAL(10,2) NULL,
    width DECIMAL(10,2) NULL,
    height DECIMAL(10,2) NULL,
    parcel_type ENUM('DOCUMENT','PACKAGE','FRAGILE','OTHER') NOT NULL DEFAULT 'PACKAGE',
    payment_type ENUM('PREPAID','COD') NOT NULL DEFAULT 'PREPAID',
    cod_amount DECIMAL(12,2) NOT NULL DEFAULT 0,
    delivery_fee DECIMAL(12,2) NOT NULL DEFAULT 0,
    status ENUM(
        'CREATED',
        'PICKED_UP',
        'IN_TRANSIT',
        'AT_HUB',
        'OUT_FOR_DELIVERY',
        'DELIVERED',
        'FAILED',
        'CANCELLED',
        'RETURNED'
    ) NOT NULL DEFAULT 'CREATED',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uq_parcels_tracking_number (tracking_number),
    KEY idx_parcels_sender (sender_customer_id),
    KEY idx_parcels_receiver (receiver_customer_id),
    KEY idx_parcels_origin_hub (origin_hub_id),
    KEY idx_parcels_destination_hub (destination_hub_id),
    KEY idx_parcels_current_hub (current_hub_id),
    KEY idx_parcels_status (status),
    KEY idx_parcels_created_at (created_at),
    CONSTRAINT fk_parcels_sender
        FOREIGN KEY (sender_customer_id) REFERENCES customers(id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_parcels_receiver
        FOREIGN KEY (receiver_customer_id) REFERENCES customers(id)
        ON UPDATE CASCADE ON DELETE SET NULL,
    CONSTRAINT fk_parcels_origin_hub
```

```
        FOREIGN KEY (origin_hub_id) REFERENCES hubs(id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_parcels_destination_hub
        FOREIGN KEY (destination_hub_id) REFERENCES hubs(id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_parcels_current_hub
        FOREIGN KEY (current_hub_id) REFERENCES hubs(id)
        ON UPDATE CASCADE ON DELETE SET NULL
);

CREATE TABLE IF NOT EXISTS parcel_items (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    parcel_id BIGINT UNSIGNED NOT NULL,
    name VARCHAR(200) NOT NULL,
    description TEXT NULL,
    quantity INT UNSIGNED NOT NULL DEFAULT 1,
    unit_price DECIMAL(12,2) NOT NULL DEFAULT 0,
    total_price DECIMAL(12,2) NOT NULL DEFAULT 0,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    KEY idx_parcel_items_parcel_id (parcel_id),
    CONSTRAINT fk_parcel_items_parcel
        FOREIGN KEY (parcel_id) REFERENCES parcels(id)
        ON UPDATE CASCADE ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS parcel_addresses (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    parcel_id BIGINT UNSIGNED NOT NULL,
    type ENUM('PICKUP','DELIVERY') NOT NULL,
    city_id BIGINT UNSIGNED NOT NULL,
    zone_id BIGINT UNSIGNED NOT NULL,
    area_id BIGINT UNSIGNED NULL,
    city_name VARCHAR(100) NOT NULL,
    zone_name VARCHAR(100) NOT NULL,
    area_name VARCHAR(100) NULL,
    address_line VARCHAR(300) NOT NULL,
    landmark VARCHAR(255) NULL,
    latitude DECIMAL(10,7) NULL,
    longitude DECIMAL(10,7) NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uq_parcel_addresses_parcel_type (parcel_id, type),
    KEY idx_parcel_addresses_city (city_id),
    KEY idx_parcel_addresses_zone (zone_id),
    KEY idx_parcel_addresses_area (area_id),
    CONSTRAINT fk_parcel_addresses_parcel
        FOREIGN KEY (parcel_id) REFERENCES parcels(id)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_parcel_addresses_city
        FOREIGN KEY (city_id) REFERENCES service_cities(id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_parcel_addresses_zone
        FOREIGN KEY (zone_id) REFERENCES service_zones(id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_parcel_addresses_area
        FOREIGN KEY (area_id) REFERENCES service_areas(id)
        ON UPDATE CASCADE ON DELETE SET NULL
);

CREATE TABLE IF NOT EXISTS parcel_drafts (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    customer_id BIGINT UNSIGNED NOT NULL,
    payload JSON NOT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uq_parcel_drafts_customer (customer_id),
    CONSTRAINT fk_parcel_drafts_customer
        FOREIGN KEY (customer_id) REFERENCES customers(id)
        ON UPDATE CASCADE ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS pickups (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    parcel_id BIGINT UNSIGNED NOT NULL,
    requested_by BIGINT UNSIGNED NULL,
    assigned_rider_id BIGINT UNSIGNED NULL,
    pickup_address VARCHAR(500) NOT NULL,
    scheduled_at DATETIME NULL,
    picked_up_at DATETIME NULL,
    status ENUM('REQUESTED','ASSIGNED','IN_PROGRESS','PICKED_UP','FAILED','CANCELLED')
        NOT NULL DEFAULT 'REQUESTED',
    failure_reason VARCHAR(500) NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
```

```
    PRIMARY KEY (id),
    KEY idx_pickups_parcel_id (parcel_id),
    KEY idx_pickups_requested_by (requested_by),
    KEY idx_pickups_rider (assigned_rider_id),
    KEY idx_pickups_status (status),
    CONSTRAINT fk_pickups_parcel
        FOREIGN KEY (parcel_id) REFERENCES parcels(id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_pickups_requested_by
        FOREIGN KEY (requested_by) REFERENCES users(id)
        ON UPDATE CASCADE ON DELETE SET NULL,
    CONSTRAINT fk_pickups_rider
        FOREIGN KEY (assigned_rider_id) REFERENCES riders(id)
        ON UPDATE CASCADE ON DELETE SET NULL
);

CREATE TABLE IF NOT EXISTS transfers (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    transfer_number VARCHAR(50) NOT NULL,
    from_hub_id BIGINT UNSIGNED NOT NULL,
    to_hub_id BIGINT UNSIGNED NOT NULL,
    route_id BIGINT UNSIGNED NULL,
    vehicle_id BIGINT UNSIGNED NULL,
    driver_id BIGINT UNSIGNED NULL,
    status ENUM('PLANNED','LOADING','IN_TRANSIT','ARRIVED','CANCELLED')
        NOT NULL DEFAULT 'PLANNED',
    departed_at DATETIME NULL,
    arrived_at DATETIME NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uq_transfers_number (transfer_number),
    KEY idx_transfers_from_hub (from_hub_id),
    KEY idx_transfers_to_hub (to_hub_id),
    KEY idx_transfers_route (route_id),
    KEY idx_transfers_vehicle (vehicle_id),
    KEY idx_transfers_driver (driver_id),
    KEY idx_transfers_status (status),
    CONSTRAINT fk_transfers_from_hub
        FOREIGN KEY (from_hub_id) REFERENCES hubs(id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_transfers_to_hub
        FOREIGN KEY (to_hub_id) REFERENCES hubs(id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_transfers_route
        FOREIGN KEY (route_id) REFERENCES routes(id)
        ON UPDATE CASCADE ON DELETE SET NULL,
    CONSTRAINT fk_transfers_vehicle
        FOREIGN KEY (vehicle_id) REFERENCES vehicles(id)
        ON UPDATE CASCADE ON DELETE SET NULL,
    CONSTRAINT fk_transfers_driver
        FOREIGN KEY (driver_id) REFERENCES users(id)
        ON UPDATE CASCADE ON DELETE SET NULL
);

CREATE TABLE IF NOT EXISTS transfer_parcels (
    transfer_id BIGINT UNSIGNED NOT NULL,
    parcel_id BIGINT UNSIGNED NOT NULL,
    loaded_at DATETIME NULL,
    unloaded_at DATETIME NULL,
    PRIMARY KEY (transfer_id, parcel_id),
    KEY idx_transfer_parcels_parcel_id (parcel_id),
    CONSTRAINT fk_transfer_parcels_transfer
        FOREIGN KEY (transfer_id) REFERENCES transfers(id)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_transfer_parcels_parcel
        FOREIGN KEY (parcel_id) REFERENCES parcels(id)
        ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE IF NOT EXISTS deliveries (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    parcel_id BIGINT UNSIGNED NOT NULL,
    hub_id BIGINT UNSIGNED NOT NULL,
    rider_id BIGINT UNSIGNED NOT NULL,
    -- Multiple rows per parcel allowed (retry after FAILED/CANCELLED)
    attempt_no INT UNSIGNED NOT NULL DEFAULT 1,
    delivery_address VARCHAR(500) NOT NULL,
    assigned_at DATETIME NULL,
    out_for_delivery_at DATETIME NULL,
    delivered_at DATETIME NULL,
    status ENUM('ASSIGNED','OUT_FOR_DELIVERY','DELIVERED','FAILED','CANCELLED','RETURNED')
        NOT NULL DEFAULT 'ASSIGNED',
    failure_reason VARCHAR(500) NULL,
    recipient_name VARCHAR(150) NULL,
    recipient_phone VARCHAR(30) NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uq_deliveries_parcel_attempt (parcel_id, attempt_no),
    KEY idx_deliveries_parcel_id (parcel_id),
    KEY idx_deliveries_hub_id (hub_id),
    KEY idx_deliveries_rider_id (rider_id),
    KEY idx_deliveries_status (status),
    CONSTRAINT fk_deliveries_parcel
        FOREIGN KEY (parcel_id) REFERENCES parcels(id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_deliveries_hub
        FOREIGN KEY (hub_id) REFERENCES hubs(id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_deliveries_rider
        FOREIGN KEY (rider_id) REFERENCES riders(id)
        ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE IF NOT EXISTS delivery_proofs (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    delivery_id BIGINT UNSIGNED NOT NULL,
    type ENUM('SIGNATURE','PHOTO','OTP','IDENTITY') NOT NULL,
    value VARCHAR(500) NULL,
    file_url VARCHAR(500) NULL,
    verified_at DATETIME NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    KEY idx_delivery_proofs_delivery_id (delivery_id),
    CONSTRAINT fk_delivery_proofs_delivery
        FOREIGN KEY (delivery_id) REFERENCES deliveries(id)
        ON UPDATE CASCADE ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS parcel_events (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    parcel_id BIGINT UNSIGNED NOT NULL,
    event_type ENUM(
        'CREATED',
        'PICKED_UP',
        'ARRIVED_HUB',
        'DEPARTED_HUB',
        'LOADED',
        'UNLOADED',
        'ASSIGNED_RIDER',
        'OUT_FOR_DELIVERY',
        'DELIVERED',
        'FAILED',
        'RETURNED'
    ) NOT NULL,
    hub_id BIGINT UNSIGNED NULL,
    user_id BIGINT UNSIGNED NULL,
    rider_id BIGINT UNSIGNED NULL,
    description VARCHAR(500) NULL,
    latitude DECIMAL(10,7) NULL,
    longitude DECIMAL(10,7) NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    KEY idx_parcel_events_parcel_time (parcel_id, created_at),
    KEY idx_parcel_events_hub_id (hub_id),
    KEY idx_parcel_events_user_id (user_id),
    KEY idx_parcel_events_rider_id (rider_id),
    CONSTRAINT fk_parcel_events_parcel
        FOREIGN KEY (parcel_id) REFERENCES parcels(id)
        ON UPDATE CASCADE ON DELETE CASCADE,
    CONSTRAINT fk_parcel_events_hub
        FOREIGN KEY (hub_id) REFERENCES hubs(id)
        ON UPDATE CASCADE ON DELETE SET NULL,
    CONSTRAINT fk_parcel_events_user
        FOREIGN KEY (user_id) REFERENCES users(id)
        ON UPDATE CASCADE ON DELETE SET NULL,
    CONSTRAINT fk_parcel_events_rider
        FOREIGN KEY (rider_id) REFERENCES riders(id)
        ON UPDATE CASCADE ON DELETE SET NULL
);

CREATE TABLE IF NOT EXISTS payments (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    parcel_id BIGINT UNSIGNED NOT NULL,
    type ENUM('DELIVERY_FEE','COD','REFUND','OTHER') NOT NULL,
    amount DECIMAL(12,2) NOT NULL,
    method ENUM('CASH','BKASH','NAGAD','CARD','BANK','ONLINE') NOT NULL,
    status ENUM('PENDING','PAID','FAILED','REFUNDED') NOT NULL DEFAULT 'PENDING',
    paid_at DATETIME NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    KEY idx_payments_parcel_id (parcel_id),
    KEY idx_payments_type_status (type, status),
```

```
    KEY idx_payments_paid_at (paid_at),
    CONSTRAINT fk_payments_parcel
        FOREIGN KEY (parcel_id) REFERENCES parcels(id)
        ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE IF NOT EXISTS settlements (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    -- Server-assigned reference (`SET-0001`), same contract as `customers.code`.
    code VARCHAR(20) NOT NULL DEFAULT '',
    customer_id BIGINT UNSIGNED NOT NULL,
    period_start DATE NOT NULL,
    period_end DATE NOT NULL,
    total_cod DECIMAL(14,2) NOT NULL DEFAULT 0,
    delivery_charges DECIMAL(14,2) NOT NULL DEFAULT 0,
    other_charges DECIMAL(14,2) NOT NULL DEFAULT 0,
    net_amount DECIMAL(14,2) NOT NULL DEFAULT 0,
    status ENUM('PENDING','PROCESSING','PAID','FAILED') NOT NULL DEFAULT 'PENDING',
    paid_at DATETIME NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uq_settlements_code (code),
    KEY idx_settlements_customer_id (customer_id),
    KEY idx_settlements_period (period_start, period_end),
    KEY idx_settlements_status (status),
    CONSTRAINT fk_settlements_customer
        FOREIGN KEY (customer_id) REFERENCES customers(id)
        ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE IF NOT EXISTS notifications (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    user_id BIGINT UNSIGNED NULL,
    customer_id BIGINT UNSIGNED NULL,
    parcel_id BIGINT UNSIGNED NULL,
    channel ENUM('SMS','EMAIL','PUSH') NOT NULL,
    event_type VARCHAR(100) NOT NULL,
    recipient VARCHAR(255) NOT NULL,
    message TEXT NOT NULL,
    status ENUM('PENDING','SENT','FAILED') NOT NULL DEFAULT 'PENDING',
    sent_at DATETIME NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    KEY idx_notifications_user_id (user_id),
    KEY idx_notifications_customer_id (customer_id),
    KEY idx_notifications_parcel_id (parcel_id),
    KEY idx_notifications_status (status),
    CONSTRAINT fk_notifications_user
        FOREIGN KEY (user_id) REFERENCES users(id)
        ON UPDATE CASCADE ON DELETE SET NULL,
    CONSTRAINT fk_notifications_customer
        FOREIGN KEY (customer_id) REFERENCES customers(id)
        ON UPDATE CASCADE ON DELETE SET NULL,
    CONSTRAINT fk_notifications_parcel
        FOREIGN KEY (parcel_id) REFERENCES parcels(id)
        ON UPDATE CASCADE ON DELETE SET NULL
);

CREATE TABLE IF NOT EXISTS support_tickets (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    customer_id BIGINT UNSIGNED NOT NULL,
    parcel_id BIGINT UNSIGNED NULL,
    assigned_to BIGINT UNSIGNED NULL,
    subject VARCHAR(255) NOT NULL,
    description TEXT NOT NULL,
    priority ENUM('LOW','MEDIUM','HIGH','URGENT') NOT NULL DEFAULT 'MEDIUM',
    status ENUM('OPEN','IN_PROGRESS','RESOLVED','CLOSED') NOT NULL DEFAULT 'OPEN',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    KEY idx_support_tickets_customer (customer_id),
    KEY idx_support_tickets_parcel (parcel_id),
    KEY idx_support_tickets_assigned_to (assigned_to),
    KEY idx_support_tickets_status_priority (status, priority),
    CONSTRAINT fk_support_tickets_customer
        FOREIGN KEY (customer_id) REFERENCES customers(id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_support_tickets_parcel
        FOREIGN KEY (parcel_id) REFERENCES parcels(id)
        ON UPDATE CASCADE ON DELETE SET NULL,
    CONSTRAINT fk_support_tickets_assigned_to
        FOREIGN KEY (assigned_to) REFERENCES users(id)
        ON UPDATE CASCADE ON DELETE SET NULL
);

CREATE TABLE IF NOT EXISTS audit_logs (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
```

```
    user_id BIGINT UNSIGNED NULL,
    action VARCHAR(100) NOT NULL,
    entity_type VARCHAR(100) NOT NULL,
    entity_id BIGINT UNSIGNED NULL,
    old_data JSON NULL,
    new_data JSON NULL,
    ip_address VARCHAR(45) NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    KEY idx_audit_logs_user_id (user_id),
    KEY idx_audit_logs_entity (entity_type, entity_id),
    KEY idx_audit_logs_created_at (created_at),
    CONSTRAINT fk_audit_logs_user
        FOREIGN KEY (user_id) REFERENCES users(id)
        ON UPDATE CASCADE ON DELETE SET NULL
);

CREATE TABLE IF NOT EXISTS sequences (
    seq_name VARCHAR(50) NOT NULL,
    next_value BIGINT UNSIGNED NOT NULL DEFAULT 0,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (seq_name)
);

SET FOREIGN_KEY_CHECKS = 1;
```

Sample INSERT statements

```
-- Sample data for re-running the project queries.
-- INSERT IGNORE makes this seed block safe to run more than once.

INSERT IGNORE INTO roles (id, name, description) VALUES
  (1, 'ADMIN', 'Full system administrator'),
  (2, 'RIDER', 'Pickup and delivery rider');

INSERT IGNORE INTO branches
  (id, name, code, phone, address, city, district, status) VALUES
  (1, 'Dhaka Central Branch', 'DHK-01', '01700000001',
   'Tejgaon, Dhaka', 'Dhaka', 'Dhaka', 'ACTIVE'),
  (2, 'Chattogram Branch', 'CTG-01', '01700000002',
   'Agrabad, Chattogram', 'Chattogram', 'Chattogram', 'ACTIVE');

INSERT IGNORE INTO hubs
  (id, branch_id, name, code, address, district, capacity, status) VALUES
  (1, 1, 'Tejgaon Hub', 'DHK-TEJ', 'Tejgaon I/A', 'Dhaka', 5000, 'ACTIVE'),
  (2, 1, 'Mirpur Hub', 'DHK-MIR', 'Mirpur 10', 'Dhaka', 2500, 'ACTIVE'),
  (3, 2, 'Agrabad Hub', 'CTG-AGR', 'Agrabad C/A', 'Chattogram', 3000, 'ACTIVE');

INSERT IGNORE INTO hub_services (hub_id, service) VALUES
  (1, 'PICKUP'), (1, 'SORTING'), (1, 'DELIVERY'),
  (2, 'PICKUP'), (2, 'DELIVERY'),
  (3, 'PICKUP'), (3, 'SORTING'), (3, 'DELIVERY');

INSERT IGNORE INTO users
  (id, branch_id, name, email, phone, password_hash, status) VALUES
  (1, 1, 'Nadia Karim', 'nadia.admin@example.com', '01800000001',
   '$2b$12$submission.placeholder.hash', 'ACTIVE'),
  (2, 1, 'Arif Hossain', 'arif.rider@example.com', '01800000002',
   '$2b$12$submission.placeholder.hash', 'ACTIVE');

INSERT IGNORE INTO user_roles (user_id, role_id) VALUES (1, 1), (2, 2);
INSERT IGNORE INTO user_hubs (user_id, hub_id) VALUES (1, 1), (2, 1);

INSERT IGNORE INTO riders
  (id, user_id, hub_id, employee_code, license_number,
   compensation_type, status) VALUES
  (1, 2, 1, 'RDR-0001', 'DL-DHK-10001', 'SALARIED', 'AVAILABLE');

INSERT IGNORE INTO customers
  (id, code, name, phone, email, type, status,
   consent_accepted_at, activated_at) VALUES
  (1, 'CUS-0001', 'Ayesha Rahman', '01900000001',
   'ayesha@example.com', 'INDIVIDUAL', 'ACTIVE', NOW(), NOW()),
  (2, 'CUS-0002', 'Tanvir Ahmed', '01900000002',
   'tanvir@example.com', 'BUSINESS', 'ACTIVE', NOW(), NOW()),
  (3, 'CUS-0003', 'Sadia Rahman', '01900000003',
   'sadia@example.com', 'INDIVIDUAL', 'ACTIVE', NOW(), NOW());

INSERT IGNORE INTO service_cities
  (id, name, code, service_type, status) VALUES
  (1, 'Dhaka', 'DHAKA', 'ISD', 'ACTIVE'),
  (2, 'Chattogram', 'CHATTOGRAM', 'OSD', 'ACTIVE');

INSERT IGNORE INTO service_zones
  (id, city_id, name, code, status) VALUES
```

```
  (1, 1, 'Dhaka North', 'NORTH', 'ACTIVE'),
  (2, 1, 'Dhaka South', 'SOUTH', 'ACTIVE'),
  (3, 2, 'Chattogram Metro', 'METRO', 'ACTIVE');

INSERT IGNORE INTO service_areas
  (id, zone_id, name, code, status) VALUES
  (1, 1, 'Mirpur', 'MIRPUR', 'ACTIVE'),
  (2, 2, 'Dhanmondi', 'DHANMONDI', 'ACTIVE'),
  (3, 3, 'Agrabad', 'AGRABAD', 'ACTIVE');

INSERT IGNORE INTO pricing_lanes
  (id, pickup_type, delivery_type, same_city, status) VALUES
  (1, 'ISD', 'SAME_CITY', 1, 'ACTIVE'),
  (2, 'ISD', 'DIFFERENT_CITY', 0, 'ACTIVE');

INSERT IGNORE INTO pricing_slabs
  (id, pricing_lane_id, min_weight_grams, max_weight_grams,
   base_fee, extra_kg_fee, cod_percentage, cod_fixed_fee, status) VALUES
  (1, 1, 0, 500, 80.00, 25.00, 1.00, 10.00, 'ACTIVE'),
  (2, 1, 501, 1000, 110.00, 25.00, 1.00, 10.00, 'ACTIVE'),
  (3, 2, 0, 500, 130.00, 35.00, 1.00, 10.00, 'ACTIVE');

INSERT IGNORE INTO parcels
  (id, tracking_number, sender_customer_id, receiver_customer_id,
   receiver_name, receiver_phone, origin_hub_id, destination_hub_id,
   current_hub_id, weight, parcel_type, payment_type, cod_amount,
   delivery_fee, status, created_at) VALUES
  (1, 'DX-20261001-001', 1, 2, 'Tanvir Ahmed', '01900000002',
   1, 3, 1, 1.20, 'PACKAGE', 'COD', 1800.00, 150.00,
   'AT_HUB', '2026-10-10 09:15:00'),
  (2, 'DX-20261002-002', 2, 1, 'Ayesha Rahman', '01900000001',
   3, 1, 1, 0.40, 'DOCUMENT', 'PREPAID', 0.00, 120.00,
   'IN_TRANSIT', '2026-10-11 11:30:00'),
  (3, 'DX-20261003-003', 1, 3, 'Sadia Rahman', '01900000003',
   1, 2, 2, 0.80, 'FRAGILE', 'COD', 2500.00, 100.00,
   'OUT_FOR_DELIVERY', '2026-10-12 14:20:00'),
  (4, 'DX-20261004-004', 3, 1, 'Ayesha Rahman', '01900000001',
   2, 1, 1, 2.10, 'PACKAGE', 'PREPAID', 0.00, 90.00,
   'DELIVERED', '2026-10-13 08:05:00'),
  (5, 'DX-20261005-005', 2, NULL, 'Mahmud Hasan', '01900000004',
   1, 3, 1, 1.75, 'PACKAGE', 'COD', 3200.00, 170.00,
   'CREATED', '2026-10-14 10:45:00');

INSERT IGNORE INTO deliveries
  (id, parcel_id, hub_id, rider_id, attempt_no, delivery_address,
   assigned_at, out_for_delivery_at, delivered_at, status) VALUES
  (1, 3, 2, 1, 1, 'Mirpur, Dhaka', '2026-10-14 08:00:00',
   '2026-10-14 09:00:00', NULL, 'OUT_FOR_DELIVERY'),
  (2, 4, 1, 1, 1, 'Tejgaon, Dhaka', '2026-10-13 09:00:00',
   '2026-10-13 10:00:00', '2026-10-13 12:15:00', 'DELIVERED');

INSERT IGNORE INTO payments
  (id, parcel_id, type, amount, method, status, paid_at) VALUES
  (1, 4, 'DELIVERY_FEE', 90.00, 'BKASH', 'PAID', '2026-10-13 12:20:00'),
  (2, 3, 'COD', 2500.00, 'CASH', 'PENDING', NULL);

INSERT IGNORE INTO support_tickets
  (id, customer_id, parcel_id, assigned_to, subject, description,
   priority, status, created_at) VALUES
  (1, 1, 1, NULL, 'Parcel delayed at hub',
   'Please confirm the next transfer time.', 'HIGH', 'OPEN',
   '2026-10-13 16:00:00');

INSERT IGNORE INTO notifications
  (id, customer_id, parcel_id, channel, event_type, recipient,
   message, status, created_at) VALUES
  (1, 1, 1, 'SMS', 'TEST_FAILURE', '01900000001',
   'Old failed test notification', 'FAILED', '2026-08-01 09:00:00'),
  (2, 3, 3, 'EMAIL', 'OUT_FOR_DELIVERY', 'sadia@example.com',
   'Your parcel is out for delivery.', 'SENT', '2026-10-14 09:05:00');
```

Section 4: Project Queries

Use the exact block format below for every query. Queries are numbered continuously and use only the categories allowed by the supplied template.

Category must be one of: DDL / INSERT / UPDATE / DELETE / SELECT-Filter / ORDER BY / LIKE / IN-BETWEEN / TOP-LIMIT / JOIN / Aggregate-GROUP BY / Subquery / Other.

Query 1

Field	Details
Category	DDL
Purpose	Create the pricing slab table with primary-key, unique, foreign-key, and CHECK constraints so invalid or overlapping weight boundaries are rejected at the database layer.

SQL:

```
CREATE TABLE IF NOT EXISTS pricing_slabs (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    pricing_lane_id BIGINT UNSIGNED NOT NULL,
    min_weight_grams INT UNSIGNED NOT NULL,
    max_weight_grams INT UNSIGNED NOT NULL,
    base_fee DECIMAL(12,2) NOT NULL DEFAULT 0,
    extra_kg_fee DECIMAL(12,2) NOT NULL DEFAULT 0,
    cod_percentage DECIMAL(5,2) NOT NULL DEFAULT 0,
    cod_fixed_fee DECIMAL(12,2) NOT NULL DEFAULT 0,
    status ENUM('ACTIVE','INACTIVE') NOT NULL DEFAULT 'ACTIVE',
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
        ON UPDATE CURRENT_TIMESTAMP,
    PRIMARY KEY (id),
    UNIQUE KEY uq_pricing_slabs_lane_min
        (pricing_lane_id, min_weight_grams),
    UNIQUE KEY uq_pricing_slabs_lane_max
        (pricing_lane_id, max_weight_grams),
    CONSTRAINT fk_pricing_slabs_lane
        FOREIGN KEY (pricing_lane_id) REFERENCES pricing_lanes(id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT chk_pricing_slabs_range
        CHECK (max_weight_grams > min_weight_grams)
);
```

Output (paste a screenshot, or a text table for SELECT queries):

Paste output screenshot here.

Expected file: `images/query-01-output.png`

Query 2

Field	Details
Category	INSERT
Purpose	Add a new active branch to extend DropX operations into Sylhet. The unique branch code prevents duplicate branch records.

SQL:

```
INSERT INTO branches
    (name, code, phone, address, city, district, status)
VALUES
    ('Sylhet Branch', 'SYL-01', '01700000003',
     'Amberkhana, Sylhet', 'Sylhet', 'Sylhet', 'ACTIVE');
```

Output (paste a screenshot, or a text table for SELECT queries):

Paste output screenshot here.

Expected file: `images/query-02-output.png`

Query 3

Field	Details
Category	UPDATE
Purpose	Move an open high-priority support ticket into progress after an operations employee starts investigating it.

SQL:

```
UPDATE support_tickets
SET status = 'IN_PROGRESS',
    assigned_to = 1
WHERE id = 1
  AND status = 'OPEN';
```

Output (paste a screenshot, or a text table for SELECT queries):

Paste output screenshot here.

Expected file: `images/query-03-output.png`

Query 4

Field	Details
Category	DELETE
Purpose	Remove failed test notifications older than 30 days so the notification queue does not retain obsolete test data.

SQL:

```
DELETE FROM notifications
WHERE status = 'FAILED'
  AND created_at < CURRENT_TIMESTAMP - INTERVAL 30 DAY;
```

Output (paste a screenshot, or a text table for SELECT queries):

Paste output screenshot here.

Expected file: `images/query-04-output.png`

Query 5

Field	Details
Category	SELECT-Filter
Purpose	Show COD parcels that are still active in the delivery lifecycle and therefore require operational attention.

SQL:

```
SELECT tracking_number, receiver_name, cod_amount, status
FROM parcels
WHERE payment_type = 'COD'
  AND status NOT IN ('DELIVERED', 'CANCELLED', 'RETURNED');
```

Output (paste a screenshot, or a text table for SELECT queries):

Paste output screenshot here.

Expected file: `images/query-05-output.png`

Query 6

Field	Details
Category	ORDER BY
Purpose	List parcels from newest to oldest, with tracking number as a deterministic secondary ordering key.

SQL:

```
SELECT tracking_number, receiver_name, status, created_at
FROM parcels
ORDER BY created_at DESC, tracking_number ASC;
```

Output (paste a screenshot, or a text table for SELECT queries):

Paste output screenshot here.

Expected file: `images/query-06-output.png`

Query 7

Field	Details
Category	LIKE
Purpose	Find customers whose names contain “Rahman” when staff only know part of the customer name.

SQL:

```
SELECT code, name, phone, email, status
FROM customers
WHERE name LIKE '%Rahman%';
```

Output (paste a screenshot, or a text table for SELECT queries):

Paste output screenshot here.

Expected file: `images/query-07-output.png`

Query 8

Field	Details
Category	IN-BETWEEN
Purpose	Find parcels in selected active statuses whose delivery fees fall within a useful operational price range.

SQL:

```
SELECT tracking_number, delivery_fee, status
FROM parcels
WHERE status IN ('CREATED', 'AT_HUB', 'OUT_FOR_DELIVERY')
  AND delivery_fee BETWEEN 80.00 AND 180.00
ORDER BY delivery_fee DESC;
```

Output (paste a screenshot, or a text table for SELECT queries):

Paste output screenshot here.

Expected file: `images/query-08-output.png`

Query 9

Field	Details
Category	TOP-LIMIT
Purpose	Display the five most recently created parcels for a compact admin dashboard widget. MySQL uses LIMIT instead of TOP.

SQL:

```
SELECT tracking_number, receiver_name, status, created_at
FROM parcels
ORDER BY created_at DESC
LIMIT 5;
```

Output (paste a screenshot, or a text table for SELECT queries):

Paste output screenshot here.

Expected file: `images/query-09-output.png`

Query 10

Field	Details
Category	JOIN
Purpose	Combine parcel, sender, origin-hub, and destination-hub data into one operations report.

SQL:

```
SELECT
    p.tracking_number,
    c.name AS sender_name,
    oh.name AS origin_hub,
    dh.name AS destination_hub,
    p.status
FROM parcels AS p
INNER JOIN customers AS c
    ON c.id = p.sender_customer_id
INNER JOIN hubs AS oh
    ON oh.id = p.origin_hub_id
INNER JOIN hubs AS dh
    ON dh.id = p.destination_hub_id
ORDER BY p.created_at DESC;
```

Output (paste a screenshot, or a text table for SELECT queries):

Paste output screenshot here.

Expected file: `images/query-10-output.png`

Query 11

Field	Details
Category	Aggregate-GROUP BY
Purpose	Summarize parcel count, total delivery fees, and total COD value for each parcel status.

SQL:

```
SELECT
    status,
    COUNT(*) AS parcel_count,
    SUM(delivery_fee) AS total_delivery_fees,
    SUM(cod_amount) AS total_cod_value
FROM parcels
GROUP BY status
ORDER BY parcel_count DESC, status ASC;
```

Output (paste a screenshot, or a text table for SELECT queries):

Paste output screenshot here.

Expected file: `images/query-11-output.png`

Query 12

Field	Details
Category	Subquery
Purpose	Identify parcels whose delivery fee is higher than the average fee across all parcels.

SQL:

```
SELECT tracking_number, delivery_fee, status
FROM parcels
WHERE delivery_fee > (
    SELECT AVG(delivery_fee)
    FROM parcels
)
ORDER BY delivery_fee DESC;
```

Output (paste a screenshot, or a text table for SELECT queries):

Paste output screenshot here.

Expected file: `images/query-12-output.png`